

第 5 章 邱奇-图灵论题

王 鑫

wangx@tju.edu.cn

天津大学 智能与计算学部



Outline

- 1 Turing Machines
- 2 Variants of Turing Machines
- 3 The Definition of Algorithm

Outline

- 1 Turing Machines
 - Formal Definition of a Turing Machine
 - Examples of Turing Machines
- 2 Variants of Turing Machines
- 3 The Definition of Algorithm

Turing Machines

Turing machine 图灵机

- first proposed by [Alan Turing](#) in 1936
- a much more powerful model than DFA/NFA and PDA
- with an unlimited and unrestricted memory
- a much more accurate model of a general purpose computer
- can do everything that a real computer can do

Even a Turing machine cannot solve certain problems

- these problems are beyond the theoretical limits of computation

艾伦·图灵 (Alan Turing)



艾伦·图灵 (Alan Turing)

June 23, 1912 – June 7, 1954 (aged 41)

英国数学家、逻辑学家

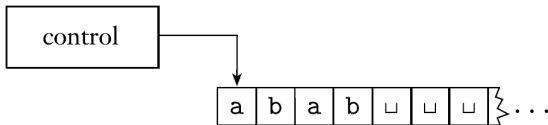
“计算机科学之父”

“人工智能之父”

Turing Machines

The Turing machine model uses an infinite tape as its unlimited memory.

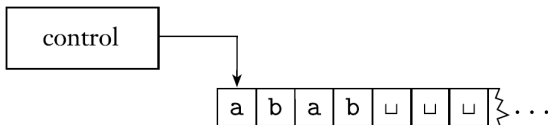
- It has a tape head that can read and write symbols and move around on the tape.
- Initially the tape contains only the input string and is blank everywhere else.
- If the machine needs to store information, it may write this information on the tape.



Turing Machines

The Turing machine model uses an infinite tape as its unlimited memory.

- To read the information that it has written, the machine can move its head back over it.
- The machine continues computing until it decides to produce an output.
- The outputs ***accept*** and ***reject*** are obtained by entering designated accepting and rejecting states.
- If it doesn't enter an accepting or a rejecting state, it will go on forever, ***never halting***.



Turing Machines

The differences between finite automata and Turing machines

- ① A Turing machine can both write on the tape and read from it.
- ② The read-write head can move both to the left and to the right.
- ③ The tape is infinite.
- ④ The special states for rejecting and accepting take effect immediately.

Turing Machines

例 (Turing machine M_1)

Let's introduce a Turing machine M_1 for testing membership in the language

$$B = \{w\#w \mid w \in \{0,1\}^*\}$$

We want M_1 to accept if its input is a member of B and to reject otherwise.

Turing Machines

例 (Turing machine M_1)

$$B = \{w\#w \mid w \in \{0,1\}^*\}$$

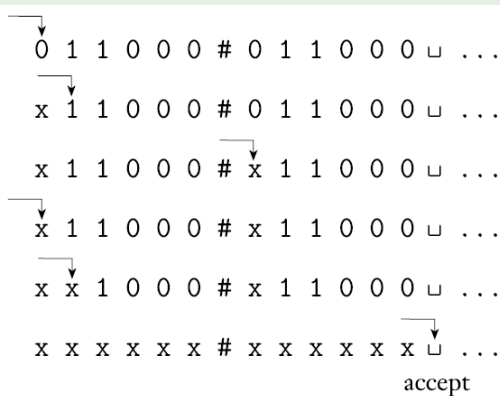
On input string w :

- 1 Zig-zag across the tape to corresponding positions on either side of the $\#$ symbol to check whether these positions contain the same symbol. If they do not, or if no $\#$ is found, **reject**. Cross off symbols as they are checked to keep track of which symbols correspond.
- 2 When all symbols to the left of the $\#$ have been crossed off, check for any remaining symbols to the right of the $\#$. If any symbols remain, **reject**; otherwise, **accept**.

Turing Machines

例 (Turing machine M_1)

several nonconsecutive snapshots of M_1 's tape after it is started on input 011000#011000.



Formal Definition of a Turing Machine

定义 (TM (图灵机))

A **Turing machine** 图灵机 (TM) is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where Q, Σ, Γ are all finite sets and

- ① Q is the set of **states**,
- ② Σ is the **input alphabet** not containing the **blank symbol** $\sqcup$,
- ③ Γ is the **tape alphabet**, where $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$,
- ④ $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the **transition function**,
- ⑤ $q_0 \in Q$ is the **start state**
- ⑥ $q_{\text{accept}} \in Q$ is the **accept state**, and
- ⑦ $q_{\text{reject}} \in Q$ is the **reject state**, where $q_{\text{accept}} \neq q_{\text{reject}}$.

Configurations of a Turing Machine

A **configuration** of the Turing machine

- the current state
- the current tape contents
- the current head location

A configuration of the Turing machine

uqv

- the current state is q
- the current tape contents is uv
- the current head location is the first symbol of v

The tape contains only blanks following the last symbol of v . $1011q701111$

Configurations of a Turing Machine

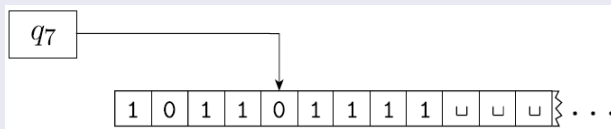
A **configuration** 格局 of the Turing machine

A configuration of the Turing machine

uqv

- the current state is q
- the current tape contents is uv
- the current head location is the first symbol of v

The tape contains only blanks following the last symbol of v . $1011q_701111$



How does a Turing machine compute

Configuration C_1 **yields** 产生 configuration C_2

- if the Turing machine can legally go from C_1 to C_2 in a single step.

Formalization

Suppose that $a, b \in \Gamma$, $u, v \in \Gamma^*$, $q_i, q_j \in Q$

- $uaq_i bv$ yields $uq_j acv$

if $\delta(q_i, b) = (q_j, c, L)$

- $uaq_i bv$ yields $uacq_j v$ if $\delta(q_i, b) = (q_j, c, R)$

How does a Turing machine compute

Configuration C_1 **yields** configuration C_2

- if the Turing machine can legally go from C_1 to C_2 in a single step.

Formalization

Special cases occur when the head is at one of the ends.

- For the left-hand end
 - left-moving: $q_i bv$ yields $q_j cv$
 - right-moving: $q_i bv$ yields $cq_j v$
- For the right-hand end
 - uaq_i is equivalent to $uaq_i\sqcup$

How does a Turing machine compute

TM M on input w

- **start configuration:** $q_0 w$
- **accepting configuration:** $\cdots q_{\text{accept}} \cdots$
- **rejecting configuration:** $\cdots q_{\text{reject}} \cdots$

Accepting and rejecting configurations are **halting configurations** and do not yield further configurations.

Because the machine is defined to halt when in q_{accept} and q_{reject} , we equivalently could have defined the transition function to have the more complicated form

- $\delta : Q' \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$, where $Q' = Q - \{q_{\text{accept}}, q_{\text{reject}}\}$

How does a Turing machine compute

A Turing machine M **accepts** input w if a sequence of configurations $C_1, C_2, \dots, C_k$ exists, where

- C_1 is the start configuration of M on input w ,
- each C_i yields C_{i+1} , and
- C_k is an accepting configuration.

The collection of strings that M accepts is ***the language of M*** , or ***the language recognized by M*** , denoted $L(M)$.

Turing-recognizable

定义 (Turing-recognizable)

Call a language ***Turing-recognizable*** 图灵可识别的 if some Turing machine recognizes it.

(It is called a ***recursively enumerable language*** 递归可枚举语言.) When we start a Turing machine on an input, three outcomes are possible.

- ① accept
- ② reject
- ③ loop

By ***loop*** we mean that the machine simply does not halt.

Turing-decidable

A Turing machine M can fail to accept an input by entering the q_{reject} state and rejecting, or by looping

- Sometimes distinguishing a machine that is looping from one that is merely taking a long time is difficult.

For this reason, we prefer Turing machines that halt on all inputs; such machines never loop.

- These machines are called ***deciders*** because they always make a decision to accept or reject.
- A decider that recognizes some language also is said to ***decide*** that language.

Turing-decidable

- These machines are called ***deciders*** because they always make a decision to accept or reject.
- A decider that recognizes some language also is said to ***decide*** that language.

定义 (Turing-decidable)

Call a language ***Turing-decidable*** 图灵可判定的 or simply ***decidable*** 可判定的 if some Turing machine decides it.

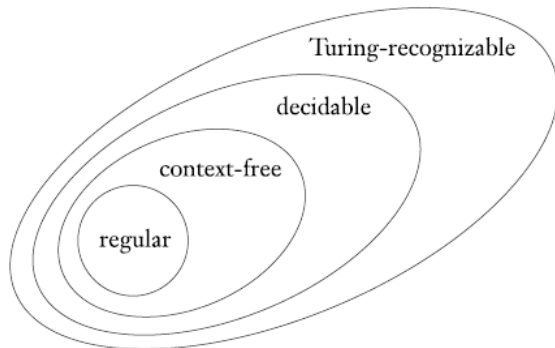
(It is called a ***recursively language*** 递归语言.)

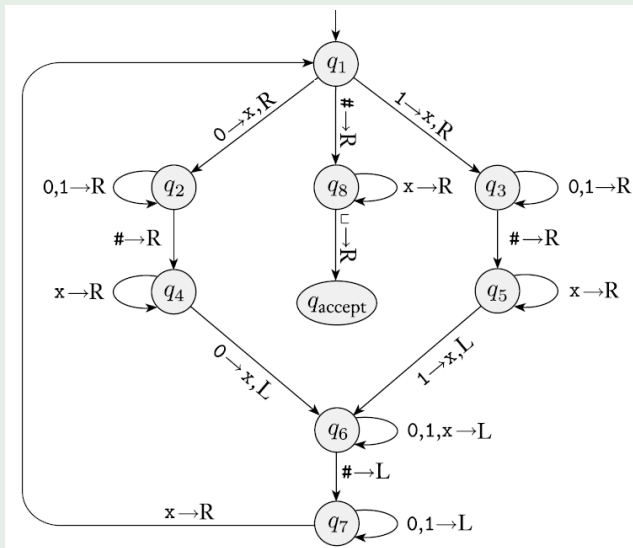
- Every decidable language is Turing-recognizable.

Relationship among the classes of languages

定理

Every context-free language is decidable.



例 (TM M_1)

Outline

1 Turing Machines

2 Variants of Turing Machines

- Multitape Turing Machines
- Nondeterministic Turing Machines 非确定型图灵机
- Enumerators
- Equivalence With Other Models

3 The Definition of Algorithm

Multitape Turing Machines 多带图灵机

定理

Every multitape Turing machine has an equivalent single-tape Turing machine.

Nondeterministic Turing Machines

- $\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$

定理

Every nondeterministic Turing machine has an equivalent deterministic Turing machine.

Enumerators 枚举器

定理

A language is Turing-recognizable if and only if some enumerator enumerates it.

Outline

- 1 Turing Machines
- 2 Variants of Turing Machines
- 3 The Definition of Algorithm
 - Hilbert's Problems

Church–Turing thesis

*Intuitive notion
of algorithms*

equals

*Turing machine
algorithms*

The Church–Turing Thesis 邱奇-图灵论题